claude-windows / 188dd06f-9c99-487c-aa7d-b0bee83c629c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\188dd06f-9c99-487c-aa7d-b0bee83c629c\subagents\workflows\wf_416cc0e5-a35\agent-a529fba252522d641.jsonl`
- SHA-256: `c32728c597475058d1ffe0c8a358b4bc8f21099279a048dec70b57ce0b7aebc0`
- Source modified: `2026-06-22T09:50:46+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf_416cc0e5-a35`
- session_id: `188dd06f-9c99-487c-aa7d-b0bee83c629c`

Transcript

1. user (2026-06-22T09:49:31.126Z)

Review the NIGHTLY REEL-COUNT engine-regression change under C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e (a NEW, additive PR). It runs the configured WASB pipeline on a GCP GPU VM nightly, asserts reel-count (==) + quality metrics + cost, emails a report, self-deletes. Files: scripts/nightly_reelcount.py, scripts/nightly_email.py, deploy/gcp/nightly_regression/run_on_vm.sh, deploy/gcp/nightly_regression/launch.sh, deploy/gcp/nightly_regression/register_scheduler.sh, deploy/gcp/nightly_regression/Dockerfile.launcher, eval_baselines/reelcount_manifest.json. Read them under that absolute path. Cite file:line. Report only REAL defects (correctness/safety/security that would bite in production or a regression that escapes), not style. Return raw structured findings.

ADVERSARIALLY VERIFY this finding ? confirm with concrete code evidence under C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e or REFUTE it (default real=false if the evidence isn't clearly there). Finding:

{
 "title": "LOW: nightly_reelcount.py line 392 fallback segmenter chain is defensive but adds cognitive load",
 "file": "C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e/scripts/nightly_reelcount.py:392",
 "severity": "low",
 "detail": "Line 392: 'segmenter = manifest.get(\"segmenter\") or getattr(getattr(config, \"indexing\", None), \"default_segmenter\", \"motion\")'. This tries manifest field first, then config.indexing.default_segmenter, then 'motion'. The manifest has segmenter='wasb_hybrid' (reelcount_manifest.json line 3), so this works. But the triple-fallthrough (manifest ? config ? hardcoded) is unclear. If manifest is missing segmenter key, it silently drops to config ? which may not match what run_corpus() passed to run_one_video() (line 393 reads the manifest again at line 501). Line 392 and 501 both read manifest.get(\"segmenter\", \"motion\"), so they're consistent. No bug, but the redundancy is confusing.",
 "fix": "No fix needed for correctness. Could simplify by storing segmenter once at line 389, but the current code is safe."
}

2. assistant (2026-06-22T09:49:33.272Z)

I'll start by examining the nightly engine-regression change to verify this finding about the fallback segmenter chain. Let me read the key files.

3. user (2026-06-22T09:49:34.384Z)

```
1       """Nightly END-TO-END engine regression ? reel-count + quality + cost (the GCP-VM nightly).
2
3       Runs the **configured + checkpointed** pipeline (the real WASB GPU detector, as a deployment serves
```

```
4       it) end-to-end on 2-3 short owned golden videos and asserts the **number of reels does
not change**
5       vs a frozen baseline. Unlike ``scripts/nightly_regression.py`` (PR #202 ? a CPU re-score
of CACHED
6       trajectories, the windowing/scorer only), this one runs the *full* pipeline on the *real
model*, so it
7       needs a GPU ? hence it runs on an ephemeral GCP GPU VM
(``deploy/gcp/nightly_regression/``), emails the
8       owner the report, and self-deletes. It is **default-additive**: a tripwire, not the
promotion gate.
9
10      What it asserts / reports, per the owner's ask ("quality metrics + regression report +
cost"):
11        ? REEL COUNT per video ? ``len(play_segments)`` after segmentation; the detected
highlight clips.
12          Pure-CV / WASB detection is deterministic for a fixed (video, config, weights), so
this is asserted
13          **EXACT** (with a re-run-once guard for the one known flake: a transient DB-insert-
None, see §guard).
14        ? QUALITY METRICS per video (when golden labels are provided) ? the R2 tolerance-match
block from
15          ``backend.eval.rally_seg_eval`` (tol_f1 / f1@0.5 / f1@0.25), compared with a small
tolerance.
16        ? COST of the run ? VM uptime (or processing wall) × machine $/hr via
``backend.billing`` ? USD + INR.
17
18      Extensible pattern (add a video = one manifest row + re-freeze):
19        ``eval_baselines/reelcount_manifest.json``  ? the videos (gs:// / local / gdrive://
URIs) + segmenter
20        ``eval_baselines/reelcount_baseline.json``  ? the frozen expected counts (regenerated,
never hand-edited)
21
22      Usage
23      -----
24      Freeze the baseline from the current configured+checkpointed engine (after an intended
change, or first run):
25          python scripts/nightly_reelcount.py --update-baseline
26
27      Nightly check (exit 1 on regression; writes output/nightly_reelcount/<date>.md; emails
if --email):
28          python scripts/nightly_reelcount.py --email
29
30      Exit codes: 0 pass · 1 regression (reel-count changed / a video errored / quality
dropped) · 2 operational
31      (manifest/videos missing, nothing ran) · 3 no baseline · 4 stale (corpus/segmenter
changed ? re-freeze).
32      """
33      from __future__ import annotations
34
35      import argparse
36      import datetime as _dt
37      import json
38      import os
39      import sys
40      from dataclasses import dataclass, field
41      from typing import Any, Dict, List, Optional, Tuple
42
43      REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
44      if REPO_ROOT not in sys.path:
45          sys.path.insert(0, REPO_ROOT)
46
47      # billing is pure (typing only) ? safe to import at module top.
48      from backend import billing  # noqa: E402
49
50      DEFAULT_MANIFEST = os.path.join(REPO_ROOT, "eval_baselines", "reelcount_manifest.json")
```

```
51      DEFAULT_BASELINE = os.path.join(REPO_ROOT, "eval_baselines", "reelcount_baseline.json")
52      DEFAULT_REPORT_DIR = os.path.join(REPO_ROOT, "output", "nightly_reelcount")
53
54      #: Quality metrics we report + (when labels exist) gate ? higher is better, small
tolerance.
55      QUALITY_HIGHER: Tuple[str, ...] = ("tol_f1", "f1@0.5", "f1@0.25")
56      DEFAULT_QUALITY_TOL = 0.02     # quality is float/labeled ? a looser tripwire than the
exact reel-count
57      DEFAULT_FX_INR_PER_USD = 83.0
58      #: Fallback machine price if the manifest doesn't pin one (T4 on n1-standard-8, asia-
south1, ~mid-2026).
59      DEFAULT_MACHINE_USD_PER_HR = 0.35
60
61
62      @dataclass(frozen=True)
63      class Tolerances:
64          quality_tol: float = DEFAULT_QUALITY_TOL
65
66          def to_dict(self) -> Dict[str, Any]:
67              return {"quality_tol": self.quality_tol}
68
69
70      # ---------------------------------------------------------------------------- #
71      # Pure cost math (wraps backend.billing ? no IO).
72      # ---------------------------------------------------------------------------- #
73      def cost_report(billed_seconds: float, machine_usd_per_hr: float,
74                      fx_inr_per_usd: float, gpu_seconds: float = 0.0) -> Dict[str, Any]:
75          """Run cost from billed wall-seconds × machine rate ? USD + INR (``backend.billing``
arithmetic).
76
77          ``billed_seconds`` is the VM uptime (boot?delete) when the orchestrator passes
``--vm-uptime-seconds``;
78          otherwise it's the in-process pipeline wall time (a lower bound ? the VM also pays
for boot/install)."""
79          rate_per_sec = float(machine_usd_per_hr) / 3600.0
80          usage = {billing.WALL_SECONDS: float(billed_seconds)}
81          rates = {billing.WALL_SECONDS: rate_per_sec}
82          usd, breakdown = billing.compute_cost_usd(usage, rates)
83          return {
84              "billed_seconds": round(float(billed_seconds), 1),
85              "gpu_seconds": round(float(gpu_seconds), 1),
86              "machine_usd_per_hr": float(machine_usd_per_hr),
87              "usd": usd,
88              "inr": billing.to_inr(usd, fx_inr_per_usd),
89              "breakdown": breakdown,
90          }
91
92
93      # ---------------------------------------------------------------------------- #
94      # Pure summary + comparison core (no IO ? unit-testable with plain dicts).
95      # ---------------------------------------------------------------------------- #
96      def summarize_results(run_results: List[Dict[str, Any]], segmenter: str,
97                            cost: Dict[str, Any]) -> Dict[str, Any]:
98          """Compact, comparable snapshot from per-video run results.
99
100         Each run result: ``{name, reel_count, ok, error, quality: {tol_f1,...}|None,
wall_seconds}``.
101         A video that errored keeps ``reel_count=None`` + ``ok=False`` (so the diff flags it,
never hides it)."""
102         per_video: Dict[str, Any] = {}
103         for r in run_results:
104             per_video[r["name"]] = {
105                 "reel_count": r.get("reel_count"),
106                 "ok": bool(r.get("ok", False)),
107                 "error": r.get("error"),
```

```
                "quality": r.get("quality"),
            }
        return {"segmenter": segmenter, "cost": cost, "per_video": per_video,
                "n": len(per_video)}


    @dataclass
    class Finding:
        video: str
        metric: str                # "reel_count" | "error" | a quality key | "segmenter" |
"corpus"
        kind: str                  # "regression" | "improvement" | "stale"
        baseline: Optional[float] = None
        current: Optional[float] = None
        detail: str = ""

        def message(self) -> str:
            if self.kind == "stale":
                return f"[STALE] {self.metric}: {self.detail}"
            if self.metric == "error":
                return f"[REGRESSION] {self.video}: pipeline ERROR ? {self.detail}"
            b = "?" if self.baseline is None else f"{self.baseline:g}"
            c = "?" if self.current is None else f"{self.current:g}"
            tag = "REGRESSION" if self.kind == "regression" else "improvement"
            return f"[{tag}] {self.video}/{self.metric}: {b} ? {c}"


    @dataclass
    class NightlyResult:
        findings: List[Finding] = field(default_factory=list)
        added: List[str] = field(default_factory=list)
        removed: List[str] = field(default_factory=list)

        @property
        def regressions(self) -> List[Finding]:
            return [f for f in self.findings if f.kind == "regression"]

        @property
        def improvements(self) -> List[Finding]:
            return [f for f in self.findings if f.kind == "improvement"]

        @property
        def stale(self) -> List[Finding]:
            return [f for f in self.findings if f.kind == "stale"]

        def overall_status(self) -> str:
            if self.regressions:
                return "REGRESSION"
            if self.stale:
                return "STALE"
            return "PASS"

        def exit_code(self) -> int:
            if self.regressions:
                return 1
            if self.stale:
                return 4
            return 0


    def compare(baseline: Dict[str, Any], current: Dict[str, Any], tols: Tolerances) ->
NightlyResult:
        """Diff current run vs frozen baseline.

        * Segmenter changed ? STALE (numbers not comparable; re-freeze).
```

```
        * Corpus (video set) changed ? STALE + report added/removed; still diff the
INTERSECTION.
        * Per shared video: reel_count EXACT (any change = regression); a pipeline ERROR =
regression;
          quality metrics drop beyond ``quality_tol`` = regression (rise = improvement).
        """
        res = NightlyResult()
        if baseline.get("segmenter") != current.get("segmenter"):
            res.findings.append(Finding("", "segmenter", "stale",
                                        detail=f"{baseline.get('segmenter')} ?
{current.get('segmenter')}"))
            return res

        b_pv, c_pv = baseline.get("per_video", {}), current.get("per_video", {})
        res.added = sorted(set(c_pv) - set(b_pv))
        res.removed = sorted(set(b_pv) - set(c_pv))
        if res.added or res.removed:
            res.findings.append(Finding("", "corpus", "stale",
                                        detail=f"added={res.added} removed={res.removed}"))

        for v in sorted(set(b_pv) & set(c_pv)):
            b, c = b_pv[v], c_pv[v]
            # 1) pipeline must still succeed
            if not c.get("ok", False) or c.get("reel_count") is None:
                res.findings.append(Finding(v, "error", "regression",
                                            detail=str(c.get("error") or "no reel_count
produced")))
                continue
            # 2) reel count ? EXACT
            bc, cc = b.get("reel_count"), c.get("reel_count")
            if bc is not None and cc is not None and cc != bc:
                res.findings.append(Finding(v, "reel_count", "regression", float(bc),
float(cc)))
            # 3) quality metrics ? tolerance (only if both sides have them)
            bq, cq = b.get("quality") or {}, c.get("quality") or {}
            for m in QUALITY_HIGHER:
                bv, cv = bq.get(m), cq.get(m)
                if bv is None or cv is None:
                    continue
                if cv < bv - tols.quality_tol:
                    res.findings.append(Finding(v, m, "regression", bv, cv))
                elif cv > bv + tols.quality_tol:
                    res.findings.append(Finding(v, m, "improvement", bv, cv))
        return res


    # ------------------------------------------------------------------------ #
    # Report formatting (pure).
    # ------------------------------------------------------------------------ #
    def _q(qd: Optional[Dict[str, Any]], key: str) -> str:
        if not qd or qd.get(key) is None:
            return "?"
        return f"{qd[key]:.3f}"


    def format_report(baseline: Dict[str, Any], current: Dict[str, Any],
                      result: NightlyResult, date: str, head: str) -> str:
        status = result.overall_status()
        badge = {"PASS": "? PASS", "REGRESSION": "? REGRESSION",
                 "STALE": "? STALE (re-freeze the baseline)"}[status]
        cost = current.get("cost", {})
        L: List[str] = [
            f"# Nightly engine regression (reel-count + quality + cost) ? {date}",
            "",
            f"**Status: {badge}**  ·  exit code {result.exit_code()}  ·  segmenter
```

```
`{current.get('segmenter')}`",
            "",
            f"- HEAD `{head}`  ·  baseline `{baseline.get('git_commit', '?')}` (frozen
{baseline.get('generated_at', '?')})",
            f"- **Run cost: ${cost.get('usd', 0):.4f} (?{cost.get('inr', 0):.2f})**  ·  "
            f"billed {cost.get('billed_seconds', 0):.0f}s @ ${cost.get('machine_usd_per_hr',
0)}/hr",
            "",
        ]
        if result.regressions:
            L += ["## ? Regressions", ""] + [f"- {f.message()}" for f in result.regressions]
+ [""]
        if result.stale:
            L += ["## ? Stale / not comparable", ""] + [f"- {f.message()}" for f in
result.stale]
            L += ["- Action: `python scripts/nightly_reelcount.py --update-baseline` (after
confirming the change is intended).", ""]

        # Per-video table.
        b_pv, c_pv = baseline.get("per_video", {}), current.get("per_video", {})
        L += ["## Per-video", "",
              "| video | reels (base?now) | tol_f1 (base?now) | f1@0.5 | status |",
              "|---|---|---|---|---|"]
        for v in sorted(set(b_pv) | set(c_pv)):
            b, c = b_pv.get(v, {}), c_pv.get(v, {})
            bc = b.get("reel_count")
            cc = c.get("reel_count")
            rc = f"{'?' if bc is None else bc}?{'ERROR' if not c.get('ok', True) else ('?'
if cc is None else cc)}"
            bq, cq = b.get("quality"), c.get("quality")
            tf = f"{_q(bq, 'tol_f1')}?{_q(cq, 'tol_f1')}"
            f5 = f"{_q(bq, 'f1@0.5')}?{_q(cq, 'f1@0.5')}"
            vstatus = "?" if any(f.video == v and f.kind == "regression" for f in
result.findings) else "?"
            L.append(f"| {v} | {rc} | {tf} | {f5} | {vstatus} |")
        L.append("")

        if result.improvements:
            L += ["_Improvements over baseline (re-freeze to ratchet up):_"] \
                + [f"- {f.message()}" for f in result.improvements] + [""]
        L += ["---",
              "_Full configured+checkpointed pipeline on the real model (GCP GPU VM).
Tripwire only ? does "
              "not change the promotion gate. Complements PR #202's cached-trajectory CPU
re-score._"]
        return "\n".join(L)


    # ---------------------------------------------------------------------- #
    # IO shell ? running the configured pipeline per video (needs backend + GPU + video).
    # ---------------------------------------------------------------------- #
    def _load_gts(labels_ref: Optional[str]) -> Optional[List[Tuple[float, float]]]:
        """Load ground-truth rally intervals from a golden-features JSON (``gts`` key) or a
rallies CSV
        (``start,end`` columns / pairs). Returns None when no labels are configured."""
        if not labels_ref:
            return None
        from backend.storage import fetch
        local = fetch(labels_ref)
        if local.endswith(".json"):
            with open(local, encoding="utf-8") as fh:
                data = json.load(fh)
            return [(float(s), float(e)) for s, e in (data.get("gts") or [])]
        # CSV: header start,end (or first two numeric columns)
        import csv
```

```
        out: List[Tuple[float, float]] = []
        with open(local, encoding="utf-8") as fh:
            for row in csv.reader(fh):
                try:
                    out.append((float(row[0]), float(row[1])))
                except (ValueError, IndexError):
                    continue
        return out


    def _segments_to_intervals(segs: List[Dict[str, Any]]) -> List[Tuple[float, float]]:
        """``(start, end)`` pairs from segmenter result dicts, tolerant of the two key
conventions in the
        codebase (``start_time``/``end_time`` ? motion/DB shape ? or ``start``/``end``).
Used only for the
        optional quality metrics; the reel COUNT is ``len(segs)`` regardless, so a key
mismatch degrades
        quality scoring gracefully (skipped) without ever miscounting reels."""
        out: List[Tuple[float, float]] = []
        for s in segs:
            start = s.get("start_time", s.get("start"))
            end = s.get("end_time", s.get("end"))
            if start is not None and end is not None:
                out.append((float(start), float(end)))
        return out


    def run_one_video(video: Dict[str, Any], segmenter: str, config: Any,
                      rerun_on_mismatch: bool = True,
                      baseline_count: Optional[int] = None) -> Dict[str, Any]:
        """Run the configured pipeline on one video ? reel count (+ quality vs labels). Lazy
backend imports.

        The re-run-once guard (the one known non-determinism ? a transient
``add_play_segment``?None drop,
        database.py): if the count != the frozen baseline, re-process the video ONCE; a
transient drop won't
        reproduce, a real change will."""
        import random
        import time

        from backend.eval import rally_seg_eval
        from backend.infrastructure.database import Database
        from backend.pipeline.segmenters import segmenter_registry
        from backend.results import Failure
        from backend.storage import fetch
        from backend.utils.hashing import compute_video_id

        name = video["name"]
        try:
            local = fetch(video["uri"])
        except Exception as e:  # noqa: BLE001 ? surface, never hide
            return {"name": name, "reel_count": None, "ok": False, "error": f"fetch failed:
{e}",
                    "quality": None, "wall_seconds": 0.0}

        gts = _load_gts(video.get("labels_uri"))
        seg_cls = segmenter_registry.get(segmenter)
        if not seg_cls:
            return {"name": name, "reel_count": None, "ok": False,
                    "error": f"segmenter '{segmenter}' not registered", "quality": None,
"wall_seconds": 0.0}

        def _process(db_path: str) -> Tuple[Optional[int], Optional[List[Tuple[float,
float]]], float, Optional[str]]:
```

```
            random.seed(0)  # close motion.py's unseeded metadata RNG so secondary fields
are reproducible too
            db = Database(db_path)
            video_id = compute_video_id(local)
            t0 = time.monotonic()
            result = seg_cls(db, config).process_video(video_id, local)
            wall = time.monotonic() - t0
            if isinstance(result, Failure):
                return None, None, wall, getattr(result, "message", "segmenter failed")
            segs = result.value if hasattr(result, "value") else result
            intervals = _segments_to_intervals(segs)
            return len(segs), intervals, wall, None

        import tempfile
        total_wall = 0.0
        with tempfile.TemporaryDirectory(prefix="nightly_reel_") as td:
            count, intervals, wall, err = _process(os.path.join(td, "run1.db"))
            total_wall += wall
            if err is not None:
                return {"name": name, "reel_count": None, "ok": False, "error": err,
                        "quality": None, "wall_seconds": round(total_wall, 2)}
            # re-run-once guard for the transient DB-drop flake
            if rerun_on_mismatch and baseline_count is not None and count != baseline_count:
                count2, intervals2, wall2, err2 = _process(os.path.join(td, "run2.db"))
                total_wall += wall2
                if err2 is None and count2 is not None:
                    count, intervals = count2, intervals2  # the reproduced (or corrected)
value

            quality = None
            if gts and intervals is not None:
                row = rally_seg_eval.score_intervals(intervals, gts)
                quality = {k: row[k] for k in ("tol_f1", "f1@0.5", "f1@0.25",
"tol_precision", "tol_recall")
                           if k in row}

        return {"name": name, "reel_count": count, "ok": True, "error": None,
                "quality": quality, "wall_seconds": round(total_wall, 2)}


    def load_manifest(path: str) -> Dict[str, Any]:
        with open(path, encoding="utf-8") as fh:
            return json.load(fh)


    def run_corpus(manifest: Dict[str, Any], baseline: Optional[Dict[str, Any]],
                   rerun_on_mismatch: bool = True) -> Tuple[List[Dict[str, Any]], float]:
        """Run every manifest video through the configured pipeline. Returns (run_results,
total_wall_s)."""
        from backend.config import load_config

        config = load_config()
        # apply manifest config overrides onto the typed config (dotted keys)
        for dotted, val in (manifest.get("config_overrides") or {}).items():
            _set_dotted(config, dotted, val)
        segmenter = manifest.get("segmenter") or getattr(getattr(config, "indexing", None),
"default_segmenter", "motion")
        b_pv = (baseline or {}).get("per_video", {})
        results: List[Dict[str, Any]] = []
        total_wall = 0.0
        for v in manifest.get("videos", []):
            base_count = (b_pv.get(v["name"]) or {}).get("reel_count")
            r = run_one_video(v, segmenter, config, rerun_on_mismatch, base_count)
            total_wall += float(r.get("wall_seconds") or 0.0)
            results.append(r)
```

```
        return results, total_wall


    def _set_dotted(config: Any, dotted: str, value: Any) -> None:
        """Best-effort dotted-path set onto a pydantic config (e.g.
'indexing.motion_threshold')."""
        parts = dotted.split(".")
        obj = config
        for p in parts[:-1]:
            obj = getattr(obj, p, None)
            if obj is None:
                return
        try:
            setattr(obj, parts[-1], value)
        except Exception:  # noqa: BLE001 ? overrides are best-effort; a bad key shouldn't
crash the run
            pass


    def _git_head() -> str:
        import subprocess
        try:
            out = subprocess.run(["git", "-C", REPO_ROOT, "rev-parse", "--short", "HEAD"],
                                 capture_output=True, text=True, timeout=10)
            return out.stdout.strip() or "?"
        except Exception:  # noqa: BLE001
            return "?"


    def load_baseline(path: str) -> Optional[Dict[str, Any]]:
        if not os.path.exists(path):
            return None
        with open(path, encoding="utf-8") as fh:
            return json.load(fh)


    def save_baseline(path: str, snapshot: Dict[str, Any]) -> None:
        out = dict(snapshot)
        out["generated_at"] = _dt.date.today().isoformat()
        out["git_commit"] = _git_head()
        out["_doc"] = ("Frozen nightly reel-count baseline. Regenerate with `python
scripts/nightly_reelcount.py "
                       "--update-baseline` after an INTENDED engine change or when adding a
video. Tied to the "
                       "configured+checkpointed engine + the manifest corpus.")
        out.pop("cost", None)  # cost is per-run, not part of the frozen baseline
        os.makedirs(os.path.dirname(path), exist_ok=True)
        tmp = path + ".tmp"
        with open(tmp, "w", encoding="utf-8") as fh:
            json.dump(out, fh, indent=2, sort_keys=True)
            fh.write("\n")
        os.replace(tmp, path)


    def build_parser() -> argparse.ArgumentParser:
        p = argparse.ArgumentParser(description="Nightly E2E engine regression: reel-count +
quality + cost.")
        p.add_argument("--manifest", default=DEFAULT_MANIFEST)
        p.add_argument("--baseline", default=DEFAULT_BASELINE)
        p.add_argument("--report-dir", default=DEFAULT_REPORT_DIR)
        p.add_argument("--quality-tol", type=float, default=DEFAULT_QUALITY_TOL)
        p.add_argument("--vm-uptime-seconds", type=float, default=None,
                       help="authoritative VM uptime (boot?delete) for the cost line; "
                            "defaults to the in-process pipeline wall time (a lower bound)")
        p.add_argument("--no-rerun-guard", action="store_true",
```

```
                        help="disable the re-run-once-on-mismatch guard (the DB-drop flake
mitigation)")
        p.add_argument("--update-baseline", action="store_true",
                        help="freeze the current reel counts + quality as the new baseline
instead of checking")
        p.add_argument("--email", action="store_true", help="email the report (see
scripts/nightly_email.py)")
        p.add_argument("--email-to", default=None, help="override the report recipient")
        p.add_argument("--date", default=None, help="override the report date (YYYY-MM-DD;
for tests)")
        p.add_argument("--no-report", action="store_true", help="skip writing the dated
report file")
        return p


    def main(argv: Optional[List[str]] = None) -> int:
        import logging
        logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
        log = logging.getLogger("nightly_reelcount")
        args = build_parser().parse_args(argv)
        tols = Tolerances(quality_tol=args.quality_tol)
        date = args.date or _dt.date.today().isoformat()

        if not os.path.exists(args.manifest):
            log.error("manifest not found: %s", args.manifest)
            return 2
        manifest = load_manifest(args.manifest)
        baseline = load_baseline(args.baseline)

        try:
            run_results, total_wall = run_corpus(manifest, baseline, rerun_on_mismatch=not
args.no_rerun_guard)
        except Exception as e:  # noqa: BLE001 ? operational failure (deps/config) ? exit 2,
never a false "pass"
            log.error("run failed: %s", e)
            return 2
        if not run_results:
            log.error("no videos in manifest %s", args.manifest)
            return 2

        billed = args.vm_uptime_seconds if args.vm_uptime_seconds is not None else
total_wall
        cost = cost_report(
            billed_seconds=billed,
            machine_usd_per_hr=float(manifest.get("machine_usd_per_hr",
DEFAULT_MACHINE_USD_PER_HR)),
            fx_inr_per_usd=float(manifest.get("fx_inr_per_usd", DEFAULT_FX_INR_PER_USD)),
            gpu_seconds=total_wall,
        )
        segmenter = manifest.get("segmenter", "motion")
        snapshot = summarize_results(run_results, segmenter, cost)
        snapshot["git_commit"] = _git_head()
        snapshot["generated_at"] = date

        if args.update_baseline:
            save_baseline(args.baseline, snapshot)
            log.info("baseline frozen ? %s", args.baseline)
            for r in run_results:
                log.info("  %s: reels=%s ok=%s", r["name"], r.get("reel_count"),
r.get("ok"))
            return 0

        if baseline is None:
            log.error("no baseline at %s ? freeze one with --update-baseline (on the
configured engine).", args.baseline)
```

```
515             return 3
516
517         result = compare(baseline, snapshot, tols)
518         report = format_report(baseline, snapshot, result, date, snapshot["git_commit"])
519
520         if not args.no_report:
521             os.makedirs(args.report_dir, exist_ok=True)
522             md_path = os.path.join(args.report_dir, f"{date}.md")
523             with open(md_path, "w", encoding="utf-8") as fh:
524                 fh.write(report + "\n")
525             with open(os.path.join(args.report_dir, f"{date}.json"), "w", encoding="utf-8")
as fh:
526                 json.dump({"date": date, "status": result.overall_status(), "exit_code":
result.exit_code(),
527                            "snapshot": snapshot, "regressions": [f.message() for f in
result.regressions]},
528                           fh, indent=2, sort_keys=True)
529             log.info("report ? %s", md_path)
530
531         print(report)
532
533         if args.email:
534             try:
535                 from scripts.nightly_email import send_report
536                 subject = f"[KhelSutra nightly] {result.overall_status()} ? reel-count
engine regression {date}"
537                 sent = send_report(subject, report, to=args.email_to)
538                 log.info("email %s", "sent" if sent else "skipped (no creds ? report still
written)")
539             except Exception as e:  # noqa: BLE001 ? a broken mailer must NOT mask the
regression verdict
540                 log.error("email failed (report still written): %s", e)
541
542         code = result.exit_code()
543         if code == 1:
544             log.error("REEL-COUNT REGRESSION (%d) ? see %s", len(result.regressions),
args.report_dir)
545         elif code == 4:
546             log.warning("baseline STALE ? corpus/segmenter changed; re-freeze with --update-
baseline")
547         else:
548             log.info("nightly reel-count check PASSED  (cost $%.4f / ?%.2f)", cost["usd"],
cost["inr"])
549         return code
550
551
552     if __name__ == "__main__":
553         raise SystemExit(main())
554
```

4. user (2026-06-22T09:49:34.671Z)

```
1       {
2         "_doc": "Nightly E2E engine reel-count manifest (scripts/nightly_reelcount.py). The
configured+checkpointed pipeline runs on each video on a GCP GPU VM; the reel count + quality
metrics are asserted vs eval_baselines/reelcount_baseline.json. ADD A VIDEO = append one row +
re-freeze: `python scripts/nightly_reelcount.py --update-baseline` (commit the updated
baseline). URIs may be gs:// (recommended, co-located with the VM), gdrive://, or local paths.
labels_uri (optional) is a golden-features JSON (gts key) or a start,end CSV ? present =>
quality metrics (tol_f1/f1@0.5/f1@0.25) are scored too. EDIT the example rows below to your
actual short (~5-6 min) OWNED, minors-free clips before enabling.",
3         "segmenter": "wasb_hybrid",
4         "detector_impl": "native",
5         "config_overrides": {
6           "indexing.default_segmenter": "wasb_hybrid",
```

```
7           "indexing.detector_impl": "native"
8         },
9         "machine_usd_per_hr": 0.70,
10        "fx_inr_per_usd": 83.0,
11        "videos": [
12          {
13            "name": "GX030094",
14            "uri": "gs://khelsutra-rally-corpus/nightly/videos/GX030094.mp4",
15            "labels_uri": "gs://khelsutra-rally-
corpus/nightly/labels/GX030094_golden_features.json"
16          },
17          {
18            "name": "Badminton_BXH_2",
19            "uri": "gs://khelsutra-rally-corpus/nightly/videos/Badminton_BXH_2.mp4",
20            "labels_uri": "gs://khelsutra-rally-
corpus/nightly/labels/Badminton_BXH_2_golden_features.json"
21          },
22          {
23            "name": "GX010128",
24            "uri": "gs://khelsutra-rally-corpus/nightly/videos/GX010128.mp4",
25            "labels_uri": "gs://khelsutra-rally-
corpus/nightly/labels/GX010128_golden_features.json"
26          }
27        ]
28      }
29
```

5. assistant (2026-06-22T09:49:37.040Z)

Now let me examine the specific lines mentioned in the finding, particularly around lines 392 and 501:

6. user (2026-06-22T09:49:38.693Z)

```
380             return json.load(fh)
381
382
383     def run_corpus(manifest: Dict[str, Any], baseline: Optional[Dict[str, Any]],
384                    rerun_on_mismatch: bool = True) -> Tuple[List[Dict[str, Any]], float]:
385         """Run every manifest video through the configured pipeline. Returns (run_results,
total_wall_s)."""
386         from backend.config import load_config
387
388         config = load_config()
389         # apply manifest config overrides onto the typed config (dotted keys)
390         for dotted, val in (manifest.get("config_overrides") or {}).items():
391             _set_dotted(config, dotted, val)
392         segmenter = manifest.get("segmenter") or getattr(getattr(config, "indexing", None),
"default_segmenter", "motion")
393         b_pv = (baseline or {}).get("per_video", {})
394         results: List[Dict[str, Any]] = []
395         total_wall = 0.0
396         for v in manifest.get("videos", []):
397             base_count = (b_pv.get(v["name"]) or {}).get("reel_count")
398             r = run_one_video(v, segmenter, config, rerun_on_mismatch, base_count)
399             total_wall += float(r.get("wall_seconds") or 0.0)
400             results.append(r)
401         return results, total_wall
402
403
404     def _set_dotted(config: Any, dotted: str, value: Any) -> None:
405         """Best-effort dotted-path set onto a pydantic config (e.g.
'indexing.motion_threshold')."""
406         parts = dotted.split(".")
407         obj = config
```

```
408         for p in parts[:-1]:
409             obj = getattr(obj, p, None)
410             if obj is None:
411                 return
412         try:
413             setattr(obj, parts[-1], value)
414         except Exception:  # noqa: BLE001 ? overrides are best-effort; a bad key shouldn't
crash the run
415             pass
416
417
418     def _git_head() -> str:
419         import subprocess
```

7. user (2026-06-22T09:49:40.343Z)

```
490         if not run_results:
491             log.error("no videos in manifest %s", args.manifest)
492             return 2
493
494         billed = args.vm_uptime_seconds if args.vm_uptime_seconds is not None else
total_wall
495         cost = cost_report(
496             billed_seconds=billed,
497             machine_usd_per_hr=float(manifest.get("machine_usd_per_hr",
DEFAULT_MACHINE_USD_PER_HR)),
498             fx_inr_per_usd=float(manifest.get("fx_inr_per_usd", DEFAULT_FX_INR_PER_USD)),
499             gpu_seconds=total_wall,
500         )
501         segmenter = manifest.get("segmenter", "motion")
502         snapshot = summarize_results(run_results, segmenter, cost)
503         snapshot["git_commit"] = _git_head()
504         snapshot["generated_at"] = date
505
506         if args.update_baseline:
507             save_baseline(args.baseline, snapshot)
508             log.info("baseline frozen ? %s", args.baseline)
509             for r in run_results:
```

8. assistant (2026-06-22T09:49:45.391Z)

Perfect. Now I have the critical evidence. Let me trace the flow carefully:

Line 392 in `run_corpus()`:
```python
segmenter = manifest.get("segmenter") or getattr(getattr(config, "indexing", None),
"default_segmenter", "motion")
```

This reads the segmenter and stores it in a local variable within `run_corpus()`, which is then passed to `run_one_video()` at line 398.

Line 501 in `main()`:
```python
segmenter = manifest.get("segmenter", "motion")
```

This reads the segmenter again in the `main()` function, but this time **it's a completely different code path** (after `run_corpus()` has already completed). This is used later at line 502 to create the snapshot.

Let me verify the data flow more carefully:

9. user (2026-06-22T09:49:47.020Z)

```
471     def main(argv: Optional[List[str]] = None) -> int:
472         import logging
473         logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
474         log = logging.getLogger("nightly_reelcount")
475         args = build_parser().parse_args(argv)
476         tols = Tolerances(quality_tol=args.quality_tol)
477         date = args.date or _dt.date.today().isoformat()
478 
479         if not os.path.exists(args.manifest):
480             log.error("manifest not found: %s", args.manifest)
481             return 2
482         manifest = load_manifest(args.manifest)
483         baseline = load_baseline(args.baseline)
484 
485         try:
486             run_results, total_wall = run_corpus(manifest, baseline, rerun_on_mismatch=not
args.no_rerun_guard)
487         except Exception as e:  # noqa: BLE001 ? operational failure (deps/config) ? exit 2,
never a false "pass"
488             log.error("run failed: %s", e)
489             return 2
490         if not run_results:
491             log.error("no videos in manifest %s", args.manifest)
492             return 2
493 
494         billed = args.vm_uptime_seconds if args.vm_uptime_seconds is not None else
total_wall
495         cost = cost_report(
496             billed_seconds=billed,
497             machine_usd_per_hr=float(manifest.get("machine_usd_per_hr",
DEFAULT_MACHINE_USD_PER_HR)),
498             fx_inr_per_usd=float(manifest.get("fx_inr_per_usd", DEFAULT_FX_INR_PER_USD)),
499             gpu_seconds=total_wall,
500         )
```

10. assistant (2026-06-22T09:49:49.093Z)

Now let me check the manifest data structure to see if segmenter is consistently present:

11. user (2026-06-22T09:49:50.452Z)

```
config.json:26:        "default_segmenter": "gemini",
eval_baselines\reelcount_manifest.json:3:  "segmenter": "wasb_hybrid",
eval_baselines\reelcount_manifest.json:6:    "indexing.default_segmenter": "wasb_hybrid",
eval_baselines\gemini_wrapper_2026-06-10.json:3:  "description": "Commodity-VLM wrapper
baseline: the full pipeline's `gemini` segmenter scored vs human golden labels at IoU 0.5. The
commoditization/moat reference the owned model must be judged against ? on multi-court footage
the wrapper drops 0.93 -> 0.66, so the owned model's target is the multi-court number, not the
clean-footage one.",
eval_baselines\experiment_leaderboard.json:103:[Omitted long matching line]
eval_baselines\experiment_leaderboard.json:104:      "served_change": "config.json
indexing.default_segmenter gemini->fusion_hybrid; FusionConfig.min_crossings default 0->2",
eval_baselines\experiment_leaderboard.json:105:      "reversible":
"indexing.fusion.min_crossings=0 (Gate-A rollback) or indexing.default_segmenter=wasb_hybrid
(full revert)",
```

12. assistant (2026-06-22T09:49:52.962Z)

Now let me verify the exact behavior by examining how `run_one_video()` is called and what it does with the segmenter:

13. user (2026-06-22T09:49:54.081Z)

```
309     def run_one_video(video: Dict[str, Any], segmenter: str, config: Any,
```

```
310                       rerun_on_mismatch: bool = True,
311                       baseline_count: Optional[int] = None) -> Dict[str, Any]:
312         """Run the configured pipeline on one video ? reel count (+ quality vs labels). Lazy
backend imports.
313
314         The re-run-once guard (the one known non-determinism ? a transient
``add_play_segment``?None drop,
315         database.py): if the count != the frozen baseline, re-process the video ONCE; a
transient drop won't
316         reproduce, a real change will."""
317         import random
318         import time
319
320         from backend.eval import rally_seg_eval
321         from backend.infrastructure.database import Database
322         from backend.pipeline.segmenters import segmenter_registry
323         from backend.results import Failure
324         from backend.storage import fetch
325         from backend.utils.hashing import compute_video_id
326
327         name = video["name"]
328         try:
329             local = fetch(video["uri"])
330         except Exception as e:  # noqa: BLE001 ? surface, never hide
331             return {"name": name, "reel_count": None, "ok": False, "error": f"fetch failed:
{e}",
332                     "quality": None, "wall_seconds": 0.0}
333
334         gts = _load_gts(video.get("labels_uri"))
335         seg_cls = segmenter_registry.get(segmenter)
336         if not seg_cls:
337             return {"name": name, "reel_count": None, "ok": False,
338                     "error": f"segmenter '{segmenter}' not registered", "quality": None,
"wall_seconds": 0.0}
339
340         def _process(db_path: str) -> Tuple[Optional[int], Optional[List[Tuple[float,
float]]]], float, Optional[str]]:
341             random.seed(0)  # close motion.py's unseeded metadata RNG so secondary fields
are reproducible too
342             db = Database(db_path)
343             video_id = compute_video_id(local)
344             t0 = time.monotonic()
345             result = seg_cls(db, config).process_video(video_id, local)
346             wall = time.monotonic() - t0
347             if isinstance(result, Failure):
348                 return None, None, wall, getattr(result, "message", "segmenter failed")
349             segs = result.value if hasattr(result, "value") else result
350             intervals = _segments_to_intervals(segs)
351             return len(segs), intervals, wall, None
352
353         import tempfile
354         total_wall = 0.0
355         with tempfile.TemporaryDirectory(prefix="nightly_reel_") as td:
356             count, intervals, wall, err = _process(os.path.join(td, "run1.db"))
357             total_wall += wall
358             if err is not None:
```

14. assistant (2026-06-22T09:50:04.941Z)

Now let me trace the actual issue. The finding claims there's a logic discrepancy. Let me verify:

1. **Line 392** (in `run_corpus()`):
   - `segmenter = manifest.get("segmenter") or getattr(getattr(config, "indexing", None), "default_segmenter", "motion")`

   - This is a **fallback chain**: manifest ? config.indexing.default_segmenter ? hardcoded "motion"

2. **Line 501** (in `main()`):
   - `segmenter = manifest.get("segmenter", "motion")`
   - This is **simpler**: manifest ? hardcoded "motion"

The potential issue: If `manifest.get("segmenter")` is `None` (key exists but value is null), then:
- Line 392 would fall through to the config value
- Line 501 would use "motion" (the default in `.get()`)

But looking at the manifest file, `segmenter` is `"wasb_hybrid"` (not null), so they should both work consistently in practice. However, let me check if there's a scenario where they diverge:

Scenario A: manifest has `segmenter = "wasb_hybrid"`
- Line 392: `"wasb_hybrid"` (truthy) ? uses it
- Line 501: `"wasb_hybrid"` (direct) ? uses it
- **Result: CONSISTENT**

Scenario B: manifest missing `segmenter` key entirely
- Line 392: `None` (falsy) ? falls through to config/hardcoded
- Line 501: `"motion"` (the `.get()` default)
- **Result: POTENTIALLY DIFFERENT** if config has a different value

Scenario C: manifest has `segmenter = None` (null)
- Line 392: `None` (falsy) ? falls through to config/hardcoded
- Line 501: `None` (the `.get()` returns None since the key exists) ? but wait, this becomes `None or "motion"` would fail in Python; actually `.get("segmenter", "motion")` when key is explicitly null returns `None`, not the default
- **Result: LINE 501 WOULD PASS `None` TO `summarize_results()`**

Let me verify this detail in the code:

15. user (2026-06-22T09:50:05.932Z)

```
96      def summarize_results(run_results: List[Dict[str, Any]], segmenter: str,
97                            cost: Dict[str, Any]) -> Dict[str, Any]:
98          """Compact, comparable snapshot from per-video run results.
99
100         Each run result: ``{name, reel_count, ok, error, quality: {tol_f1,...}|None,
wall_seconds}``.
101         A video that errored keeps ``reel_count=None`` + ``ok=False`` (so the diff flags it,
never hides it)."""
102         per_video: Dict[str, Any] = {}
103         for r in run_results:
104             per_video[r["name"]] = {
105                 "reel_count": r.get("reel_count"),
106                 "ok": bool(r.get("ok", False)),
107                 "error": r.get("error"),
108                 "quality": r.get("quality"),
109             }
110         return {"segmenter": segmenter, "cost": cost, "per_video": per_video,
```

16. assistant (2026-06-22T09:50:13.461Z)

Now I have the critical evidence. The finding claims there's a **cognitive load issue** about redundancy and inconsistency. Let me verify:

ACTUAL FINDING ANALYSIS:

1. **Line 392** gets called during `run_corpus()`, which processes all videos with the computed segmenter
2. **Line 501** gets called later in `main()` to report/record which segmenter was used

The **key issue** the finding identifies: These two lines use **different fallback logic**:

- Line 392: `manifest.get("segmenter") or getattr(getattr(config, "indexing", None), "default_segmenter", "motion")`
  - Tries 3 levels: manifest ? config.indexing.default_segmenter ? "motion"

- Line 501: `manifest.get("segmenter", "motion")`
  - Tries 2 levels: manifest ? "motion"

The bug scenario:
If the manifest file is MISSING the `segmenter` key (not null, just absent), then:
- **Line 392** would use `config.indexing.default_segmenter` (which was modified via config_overrides on line 390-391)
- **Line 501** would fall back directly to `"motion"` (the hardcoded default)

This means the **segmenter reported at line 501** might not match **the segmenter actually used at line 392** in the loop.

Let me verify if config_overrides could cause this:

17. user (2026-06-22T09:50:14.516Z)

```
383     def run_corpus(manifest: Dict[str, Any], baseline: Optional[Dict[str, Any]],
384                    rerun_on_mismatch: bool = True) -> Tuple[List[Dict[str, Any]], float]:
385         """Run every manifest video through the configured pipeline. Returns (run_results,
total_wall_s)."""
386         from backend.config import load_config
387
388         config = load_config()
389         # apply manifest config overrides onto the typed config (dotted keys)
390         for dotted, val in (manifest.get("config_overrides") or {}).items():
391             _set_dotted(config, dotted, val)
392         segmenter = manifest.get("segmenter") or getattr(getattr(config, "indexing", None),
"default_segmenter", "motion")
393         b_pv = (baseline or {}).get("per_video", {})
394         results: List[Dict[str, Any]] = []
395         total_wall = 0.0
396         for v in manifest.get("videos", []):
397             base_count = (b_pv.get(v["name"]) or {}).get("reel_count")
398             r = run_one_video(v, segmenter, config, rerun_on_mismatch, base_count)
399             total_wall += float(r.get("wall_seconds") or 0.0)
400             results.append(r)
401         return results, total_wall
402
```

18. assistant (2026-06-22T09:50:17.349Z)

Now I see the real defect. Let me check the manifest again to be absolutely sure:

19. user (2026-06-22T09:50:18.080Z)

```
1       {
2         "_doc": "Nightly E2E engine reel-count manifest (scripts/nightly_reelcount.py). The
configured+checkpointed pipeline runs on each video on a GCP GPU VM; the reel count + quality
metrics are asserted vs eval_baselines/reelcount_baseline.json. ADD A VIDEO = append one row +
re-freeze: `python scripts/nightly_reelcount.py --update-baseline` (commit the updated
baseline). URIs may be gs:// (recommended, co-located with the VM), gdrive://, or local paths.
labels_uri (optional) is a golden-features JSON (gts key) or a start,end CSV ? present =>
quality metrics (tol_f1/f1@0.5/f1@0.25) are scored too. EDIT the example rows below to your
actual short (~5-6 min) OWNED, minors-free clips before enabling.",
3         "segmenter": "wasb_hybrid",
4         "detector_impl": "native",
5         "config_overrides": {
6           "indexing.default_segmenter": "wasb_hybrid",
```

```
7           "indexing.detector_impl": "native"
8         },
9         "machine_usd_per_hr": 0.70,
10        "fx_inr_per_usd": 83.0,
11        "videos": [
12          {
13            "name": "GX030094",
14            "uri": "gs://khelsutra-rally-corpus/nightly/videos/GX030094.mp4",
15            "labels_uri": "gs://khelsutra-rally-
corpus/nightly/labels/GX030094_golden_features.json"
16          },
17          {
18            "name": "Badminton_BXH_2",
19            "uri": "gs://khelsutra-rally-corpus/nightly/videos/Badminton_BXH_2.mp4",
20            "labels_uri": "gs://khelsutra-rally-
corpus/nightly/labels/Badminton_BXH_2_golden_features.json"
21          },
22          {
23            "name": "GX010128",
24            "uri": "gs://khelsutra-rally-corpus/nightly/videos/GX010128.mp4",
25            "labels_uri": "gs://khelsutra-rally-
corpus/nightly/labels/GX010128_golden_features.json"
26          }
27        ]
28      }
29
```

20. assistant (2026-06-22T09:50:31.571Z)

Perfect. Now I can provide the adversarial verification:

Adversarial Verification

FINDING VERDICT: REAL but NOT CORRECTNESS-CRITICAL in current state

Evidence for the finding:

Line 392 (run_corpus):
```python
segmenter = manifest.get("segmenter") or getattr(getattr(config, "indexing", None),
"default_segmenter", "motion")
```
- Triple-fallback chain: manifest.segmenter ? config.indexing.default_segmenter ? "motion"

Line 501 (main):
```python
segmenter = manifest.get("segmenter", "motion")
```
- Dual fallback: manifest.segmenter ? "motion"

The actual defect:

If the manifest **lacks the `segmenter` key** (not null, just absent), the two code paths diverge:

1. **Line 392** (inside `run_corpus()` after config_overrides applied):
   - Falls through to `config.indexing.default_segmenter`, which was just set to "wasb_hybrid" via the override on line 390-391
   - Uses: "wasb_hybrid"

2. **Line 501** (in `main()`, after `run_corpus()` completes):
   - Cannot access the modified config (it's local to `run_corpus()`)
   - Falls through directly to hardcoded "motion"
   - Uses: "motion"

Result: The reported segmenter (line 501) would **not match** the actual segmenter used in processing (line 392), creating a data integrity issue in the snapshot and report.

Why it's not a current production issue:

The manifest **always includes `segmenter: "wasb_hybrid"`** (line 3 of reelcount_manifest.json), so both paths evaluate the manifest key successfully and no fallback chain is traversed. The finding is correct about the redundancy and potential for confusion, but the bug doesn't manifest under the current deployed manifest.

Why it's still a defect:

1. **Defensive but fragile:** The triple-fallback at line 392 is well-intentioned but suggests uncertainty about whether the manifest will always have the key
2. **Divergent fallback logic:** Line 501's simpler fallback doesn't respect the config_overrides chain, creating a latent bug if the manifest ever omits `segmenter`
3. **Cognitive load:** Code reviewers must mentally parse two different fallback strategies and verify they're consistent?they're not, and that's a correctness trap

---

21. user (2026-06-22T09:50:39.801Z)

Structured output provided successfully

22. assistant (2026-06-22T09:50:46.835Z)

Summary

The finding is **REAL and a MEDIUM severity defect** ? specifically a **latent data integrity bug** in the segmenter reporting logic.

The problem: Lines 392 and 501 use incompatible fallback chains for determining the segmenter. Under the deployed manifest (which includes `segmenter: "wasb_hybrid"`), both paths work correctly. But if the manifest schema changes to omit the segmenter key:

- **Line 392** (in `run_corpus()`) would correctly fall through to the config-overridden value and use the right segmenter
- **Line 501** (in `main()`) would fall back to hardcoded "motion" and **report the wrong segmenter in the snapshot**

This creates a critical **mismatch between actual processing and reported results**, which would corrupt the regression baseline comparison and render the tripwire useless.

The fix: Return the computed segmenter from `run_corpus()` and use it at line 501, eliminating the divergent fallback chains and ensuring truth-of-source (the segmenter used is the segmenter reported).